

Assignment 2

Julia Nething
nething@gatech.edu

1 RESEARCH LOG

1.1 Background

Reflecting on last week's assignment, I mostly attempted to learn about the current state of how recursion is taught to novice programmers. I teach at a software engineering immersive (Hack Reactor) and I have found that algorithms, in particular problems using recursion, are the hardest to teach. This week, I would like to branch out a bit into other tangential topics. For example, my mentor suggested I look into John Hattie and effect size. My peers (in the peer reviews) suggested that I look into how different learning styles affect teaching strategies. My coworker suggested that I learn more about interleaving, because that could be an interesting way to teach multiple algorithm strategies instead of focusing solely on recursion. I think it would be interesting to see how other algorithm strategies are taught in case they are different from how teachers approach recursion. There other related topics that could help students learn recursion better, such as debugging skills or crafting effective feedback. Finally, I would like to look into more meta research papers such as how to design effective pre and post tests. While I think that the larger topic of teaching recursion is still important to me so that I can better help my students, I wonder if I should focus on underlying skills such as debugging and algorithm design that will help students on a deeper level.

1.2 Papers

1.2.1 *Has John Hattie really found the holy grail of research on teaching? An extended review of Visible Learning*

Ewald Terhart (2011). "Has John Hattie really found the holy grail of research on teaching? An extended review of Visible Learning". In: *Journal of Curriculum Studies* 43:3, pp. 425–438. DOI: [10.1080/00220272.2011.576774](https://doi.org/10.1080/00220272.2011.576774). eprint: <https://doi.org/10.1080/00220272.2011.576774>. URL: <https://doi.org/10.1080/00220272.2011.576774>

I found this article via Google Scholar after searching for "John Hattie" at the

suggestion of my mentor.

The author of this paper critiques Hattie's book *Visible Learning* and summarizes key points. Much of the critique stems in the author's opinion that the data was aggregated in a way that is not accurate for determining meaningful information. The author also summarizes Hattie's view that learning should be based in students and teachers continuously taking each others' perspectives into account (the teacher should see the material in the eyes of the student and the student should teach themselves material) (Terhart, 2011).

I have not read Hattie's book due to time constraints, so it was interesting to read a critique of his book and basic summary. It seems that the author's main objection is how Hattie combined various research studies, which seems like a fair critique. I would be interested to read further papers that either praise Hattie's work or implement his findings in a real classroom.

1.2.2 *The Power of Feedback*

John Hattie and Helen Timperley (2007). "The Power of Feedback". In: *Review of Educational Research* 77.1, pp. 81–112. DOI: [10.3102/003465430298487](https://doi.org/10.3102/003465430298487). eprint: <https://doi.org/10.3102/003465430298487>. URL: <https://doi.org/10.3102/003465430298487>

I found this article via Google Scholar after searching for "John Hattie" at the suggestion of my mentor.

The authors looked at 196 studies on the effect of feedback in classrooms and found that feedback that included suggestions for how to improve was more helpful than simple positive or negative reinforcement. The three feedback questions that the authors identify are "where am I going", "how am I going", and "where to next" (Hattie and Timperley, 2007). They also identify four types of feedback: "task", "process", "self-regulation", and "the self (person)" (Hattie and Timperley, 2007).

I enjoyed reading this paper because feedback is such an important part of the learning process, and yet I have never seen it broken down into various components like this. The authors also mention that there has been a lot of research into the timing of feedback and how certain types of feedback should be shared immediately versus shared later, so I would like to look into that further (Hattie and Timperley, 2007). This paper is relevant for teaching programming because

within a coding exercise we can give feedback related to both the code and the students' overall processes for approaching the code and debugging.

1.2.3 *Learning Styles and Performance in the Introductory Programming Sequence*

Lynda Thomas et al. (Feb. 2002). "Learning Styles and Performance in the Introductory Programming Sequence". In: *SIGCSE Bull.* 34.1, pp. 33–37. ISSN: 0097-8418. DOI: [10.1145/563517.563352](https://doi.org/10.1145/563517.563352). URL: <https://doi.org/10.1145/563517.563352>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "learning styles programming" at the suggestion of peer review feedback.

The authors first summarize many different ways of categoring learning styles: "left brain" (analytical) versus "right brain" (creative), seeing versus hearing versus hands-on, Myers-Briggs Type Indicator (MBTI) categories, abstract versus concrete, active learning versus reflective learning, and many more (Thomas et al., 2002). The authors gave students a test based on the Felder-Silverman scale and then used those results to compare the students at the end of the year. They found that certain groups did better on certain types of activities (for example coursework versus exams) and that overall active, sensing, and visual learners performed worse in the class (Thomas et al., 2002).

This was an interesting study because they did not change the curriculum at all, nor did they give different versions of the curriculum to different students. Instead, they tested the current version of the curriculum to see if all types of learners were being represented by the curriculum. I do wish the authors went more in-depth into what the curriculum entailed so we could understand better how the activities corresponded to the types of learning.

1.2.4 *A Gap Analysis of Statistical Data Reporting in K-12 Computing Education Research: Recommendations for Improvement*

Monica M. McGill and Adrienne Decker (2020). "A Gap Analysis of Statistical Data Reporting in K-12 Computing Education Research: Recommendations for Improvement". In: *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*. SIGCSE '20. Portland, OR, USA: Association for Computing Machinery, pp. 591–597. ISBN: 9781450367936. DOI: [10.1145/3328778.3366842](https://doi.org/10.1145/3328778.3366842).

URL: <https://doi.org/10.1145/3328778.3366842>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "effect size" at the suggestion of my mentor.

The authors wanted to determine how effectively computing education research papers conform to common research standards. They first compared standards from four major research entities (for example the American Psychological Association) to determine what standards are common. They then grouped those standards into categories such as methodology, statistics, and interpretation, and compared those to 76 actual computer science research studies (McGill and Decker, 2020). They found that almost all of those studies did not meet the standards for major research organizations (for example many of these papers used small sample sizes that were not repeated) (McGill and Decker, 2020).

I really enjoyed reading this paper because it confirmed what I noticed in Assignment 1, which is that the studies I was reading about used very small sample sizes and were not replicated. I also really appreciate the chart that the authors made of all the standards across the four research entities. If I do end up going down the research route, I will definitely reference that chart to make sure that I am meeting common standards.

1.2.5 *Interleaving Helps Students Distinguish among Similar Concepts*

Doug Rohrer (2012). "Interleaving Helps Students Distinguish among Similar Concepts". In: *Educational psychology review*. 24.3, pp. 355–367. ISSN: 1040-726X

I found this article via Google Scholar after searching for "interleaving learning" at the suggestion of my co-worker, who has read about the subject and thought I would be interested.

The author explains that there are two ways of presenting information to students: blocking, where each several examples of the same concept are introduced before introducing the next concept, and interleaving, where examples of several concepts are shown in various orders. Interleaving concepts leads to higher test scores because students have practice categorizing the problem before solving the problem (Rohrer, 2012). The author notes, however, that the studies referenced in this paper were not tested repeatedly long-term, and some students may struggle more with interleaving if they do not understand how to differentiate the problems (Rohrer, 2012).

I am interested in interleaving in the context of algorithm-style questions such as recursive problems. The papers that I read in Assignment 1 focused on teaching recursion by itself. However, I wonder if it would help students more if recursion was interleaved with other algorithm strategies, such as "greedy" implementations or using different types of data structures.

1.2.6 Active Learning of Greedy Algorithms by Means of Interactive Experimentation

J. Ángel Velázquez-Iturbide and Antonio Pérez-Carrasco (July 2009). "Active Learning of Greedy Algorithms by Means of Interactive Experimentation". In: *SIGCSE Bull.* 41.3, pp. 119–123. ISSN: 0097-8418. DOI: [10.1145/1595496.1562917](https://doi.org/10.1145/1595496.1562917). URL: <https://doi.org/10.1145/1595496.1562917>

I searched for "learning leetcode algorithms" on Google Scholar, which then led me to a related search of "active learning of greedy algorithms", which brought me to this paper.

The authors of this paper wanted to create interactive programs to help students actively learn greedy algorithms. First, they divided greedy algorithms into different sub-categories, then they created a different application for each sub-category. Each application shows a visualization (graph), code, and a table of values. They ran three sessions where students used the applications (total of less than 100 students), and at the end students self-reported that they enjoyed the applications and it helped them learn (Velázquez-Iturbide and Pérez-Carrasco, 2009).

This paper is very similar to one of the recursion papers that I read in Assignment 1 (they had a similar application to help students learn recursion). This one at least replicated the study with different groups of students, though the sample size was still small. Unfortunately the website where they hosted these applications is no longer a valid URL, because I would have liked to try it and see if it is something I could build off of in the next phase.

1.2.7 Dynamic programming - Structure, difficulties and teaching

E. Enström (2013). "Dynamic programming - Structure, difficulties and teaching". In: *2013 IEEE Frontiers in Education Conference (FIE)*, pp. 1857–1863. DOI: [10.1109/FIE.2013.6685158](https://doi.org/10.1109/FIE.2013.6685158)

I found this via Google Scholar by searching for "teaching dynamic programming".

The authors of this paper teach an algorithms class and wanted to pinpoint the pain points of the course. They knew from past results that dynamic programming and recursion were two sources of confusion for students. They implemented new visualizations, forms/quizzes, and a new assignment to help students understand these problems. Students self-reported that all of these were helpful but the lab was helpful to the most number of students (Enström, 2013).

I really appreciate that these authors were trying to make the most difficult parts of the class easier for students and were directly addressing them with extra practice. I also liked the reflective statements they asked students to evaluate such as "I could implement a dynprog algorithm that is presented as pseudo code, in some programming language" and "I could determine from the problem statement whether a problem should be recursively solved, without it being stated anywhere, if I got some time to investigate it" (Enström, 2013). These demonstrate nuanced layers of students' understanding and I am interested in incorporating similar questions into my future project.

1.2.8 RAPTOR: A Visual Programming Environment for Teaching Algorithmic Problem Solving

Martin C. Carlisle et al. (2005). "RAPTOR: A Visual Programming Environment for Teaching Algorithmic Problem Solving". In: *Proceedings of the 36th SIGCSE Technical Symposium on Computer Science Education*. SIGCSE '05. St. Louis, Missouri, USA: Association for Computing Machinery, pp. 176–180. ISBN: 1581139977. DOI: [10.1145/1047344.1047411](https://doi.org/10.1145/1047344.1047411). URL: <https://doi.org/10.1145/1047344.1047411>

I found this via Google Scholar by searching for papers that cited one of the earlier papers I used, (Thomas et al., 2002).

The authors of this paper wanted to teach algorithms to students without having the quirks of certain programming languages get in the way. They developed RAPTOR, which is an application that allows students to select high-level design choices (such as loops, if/else statements, etc.) without being bogged down in language-specific syntax errors. By comparing exam scores in different semesters, the authors found that students did better on algorithm-style

questions when they had RAPTOR at their disposal, but did worse on more language-specific questions like using arrays (Carlisle et al., 2005).

This was a really interesting approach and not one that I saw last week for Assignment 1 even though it seems obvious now. By removing the complexity of the particular language and focusing on the high-level functionality, students can focus on the algorithm but not get bogged down by the language details. I wonder if this could be the first step that students take, and once they make their algorithm with RAPTOR they then translate it into their language of choice (similar to pseudo-coding).

1.2.9 Introducing an Educational Tool for Learning Branch & Bound Strategy

Sofriesilero Zumaytis and Oscar Karnalim (2017). “Introducing an educational tool for learning branch & bound strategy”. In: *Journal of Information Systems Engineering and Business Intelligence* 3.1, pp. 8–15

I found this via Google Scholar by searching for papers that cited one of the earlier papers I used, (Velázquez-Iturbide and Pérez-Carrasco, 2009).

The authors found that the branch and bound strategy (used for problems such as N-Queens) is difficult for their undergraduate students to understand, so they created an application called AP-BB to help students learn. AP-BB includes a brute-force method visualization, a branch and bound method visualization, and a reduced cost matrix calculator so that students can compare all three and understand the strategies (Zumaytis and Karnalim, 2017). Twenty students self-reported that this helped them understand the branch and bound strategy (Zumaytis and Karnalim, 2017).

Branch and bound is another strategy that is related to recursion, so I was interested to see how some teachers are approaching teaching it. If I made something similar for recursion, it could compare a recursive solution visualization with an iterative solution visualization. Like most of the studies I have read about, this sample size is too small (twenty students) to know if it was useful or not but it seems interesting.

1.2.10 Pretest-Posttest Designs and Measurement of Change

Dimiter M Dimitrov and Phillip D Rumrill Jr (2003). “Pretest-posttest designs and measurement of change”. In: *Work* 20.2, pp. 159–165

I found this via Google Scholar by searching for "pre post test design".

The authors explain different ways to design your study with pre and post tests, including using random control groups, non random control groups, and using a four group design (so one experimental and one control group both do not receive the pre test). They also describe analysis of variance and covariance data and how researchers come to conclusions from that data. They state that comparing pre and post test data can actually be misleading for many reasons (Dimitrov and Rumrill Jr, 2003).

I wanted to read an article about pre and post test design to see how research studies should be conducted. Classrooms are probably very difficult places to get meaningful statistics, but they also might be useful because they can be repeated across many iterations (semesters, years, etc.). This paper was helpful to read about different ways of dividing control and experimental groups, however the second half of the paper seemed to indicate that comparing pre and post test data is extremely difficult to do with accuracy. I assume there is good research out there on how to compare pre and post test data reliably but this paper does not clearly explain how to do that.

1.2.11 *Exploring the Effects of Gender and Learning Styles on Computer Programming Performance: Implications for Programming Pedagogy*

Wilfred WF Lau and Allan HK Yuen (2009). "Exploring the effects of gender and learning styles on computer programming performance: implications for programming pedagogy". In: *British Journal of Educational Technology* 40.4, pp. 696–712

I found this via Google Scholar by searching for "learning styles programming".

In this study, the authors wanted to find out if gender and/or learning style had any effect on performance on programming assignments. They used the Gregorc style delineator to classify the students' learning styles (abstract sequential, abstract random, concrete sequential, concrete random) and then gave them test questions (Lau and Yuen, 2009). They tested 217 students and found that those with sequential learning styles (either abstract sequential or concrete sequential) did better on the test, but gender did not matter (Lau and Yuen, 2009).

It makes sense that students who have a more sequential learning style would also do well with programming because interpreters are also sequential. The

students self-reported which learning style they had, and I wonder if that is biased in any way. I would be interested to see if other personality assessments (such as the Myers-Briggs) have similar results.

1.2.12 Using Puzzles in Teaching Algorithms

Anany Levitin and Mary-Angela Papalaskari (Feb. 2002). "Using Puzzles in Teaching Algorithms". In: *SIGCSE Bull.* 34.1, pp. 292–296. ISSN: 0097-8418. DOI: [10.1145/563517.563456](https://doi.org/10.1145/563517.563456). URL: <https://doi.org/10.1145/563517.563456>

I found this via Google Scholar by searching for "teaching algorithm design".

The authors of this paper believe that a good way to teach algorithms is to have students solve puzzles that use underlying algorithmic techniques. This allows them to not focus on syntax or other language-specific issues but rather focus on the algorithmic techniques themselves. They give many examples including the Tower of Hanoi problem, the missionaries and cannibals problem, n-Queens, and more (Levitin and Papalaskari, 2002).

I was inspired by the RAPTOR paper from earlier and wanted to find similar papers where they completely take programming out of the equation. I like that this teaches students the underlying principles of different algorithms and allows them to solve problems the way they are used to (with pen and paper or with physical objects). This could be a really great way to help more "hands-on" learners understand recursion.

1.2.13 Towards a Framework for Teaching Debugging

Chen Li et al. (2019). "Towards a Framework for Teaching Debugging". In: *Proceedings of the Twenty-First Australasian Computing Education Conference*. ACE '19. Sydney, NSW, Australia: Association for Computing Machinery, pp. 79–86. ISBN: 9781450366229. DOI: [10.1145/3286960.3286970](https://doi.org/10.1145/3286960.3286970). URL: <https://doi.org/10.1145/3286960.3286970>

I found this via Google Scholar by searching for "teaching debugging".

The authors aim to teach debugging. First they categorize different types of debugging knowledge: domain knowledge, procedural knowledge, strategic knowledge, system knowledge, and previous experience (Li et al., 2019). They then explain how current debugging tools address these different types of debugging knowledge. The authors argue that no tool addresses all of these types and there

is a lack of debugging tools that do not use code at all (Li et al., 2019).

I was interested to see the different types of debugging knowledge that the authors proposed. This could be very helpful in understanding where students are confused, for example in recursive problems. I also thought it was interesting that they noted a lack of code-free debugging tools; perhaps it does make sense to teach debugging strategies in a language-agnostic way first and then have students apply that knowledge to code later.

1.2.14 *Teaching Debugging*

Robert F. Mathis (Jan. 1974). "Teaching Debugging". In: *SIGCSE Bull.* 6.1, pp. 59–63. ISSN: 0097-8418. DOI: [10.1145/953057.810443](https://doi.org/10.1145/953057.810443). URL: <https://doi.org/10.1145/953057.810443>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "teaching debugging".

The author of this paper wrote a course on debugging and this was his high-level overview of the course. He first focuses on debugging prevention, such as modularizing code. Then he goes into various tools for checking values and tracing code. Finally, he walks through some projects and case studies for students to apply this knowledge with (Mathis, 1974).

This was an interesting paper because the author notes that this is one of the first papers focused on debugging (it was written in 1974). I also like the idea of structuring your class around debugging, considering it is a huge part of each programmer's day. The focus on preventing bugs is also a good way to approach modularizing code in general and may be helpful to students.

1.2.15 *The Debugging Mindset: Understanding the Psychology of Learning Strategies Leads to Effective Problem-Solving Skills.*

Devon H. O'Dell (Feb. 2017). "The Debugging Mindset: Understanding the Psychology of Learning Strategies Leads to Effective Problem-Solving Skills." In: *Queue* 15.1, pp. 71–90. ISSN: 1542-7730. DOI: [10.1145/3055301.3068754](https://doi.org/10.1145/3055301.3068754). URL: <https://doi.org/10.1145/3055301.3068754>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "teaching debugging" and filtered by the past 5 years.

The author of this paper theorizes that learning strategies can help with the debugging process. He suggests applying a few strategies such as active recall, segmented, study, and perseverance to the debugging process (O'Dell, 2017). He also emphasizes that forming a good hypothesis for why the bug is occurring is an extremely important first step in the process (O'Dell, 2017).

I found it interesting that the author puts so much emphasis on forming a good hypothesis. Recently with my own students we implemented a debugging exercise that included forming a hypothesis about the bug before investigating. Anecdotally, I think this was a big indicator on whether or not the student would eventually find the bug. I would like to learn more about debugging hypotheses and any research there might be around that topic.

1.3 Synthesis

One of the themes that I looked into this week was pedagogy, such as viewing material in the eyes of the student and giving feedback that included suggestions to improve (Terhart, 2011) (Hattie and Timperley, 2007). I also learned about how interleaving different concepts (instead of just showing concepts linearly) was more effective for students' learning because they also learned how to identify the problem (Rohrer, 2012).

Also, I looked into how learning styles affect how educational resources are helpful to students. The students who were part of the visual, sensing, and active learning groups tended to do worse than their counterparts (Thomas et al., 2002). Another study found that sequential (as opposed to random) learners performed best (Lau and Yuen, 2009).

In addition, I looked into more a more meta aspect of research and how different educational research studies conform to research standards (McGill and Decker, 2020) (Dimitrov and Rumrill Jr, 2003). Researchers also found that pre and post test data often can be misleading (Dimitrov and Rumrill Jr, 2003). This were both very helpful in understanding what is considered a structurally sound research study, and confirmed my suspicions that many of the papers I have read so far do not hold up to rigorous scrutiny according to these standards.

I also looked into specific styles for teaching algorithms. Showing students visualizations for algorithms was a good strategy, and some of those visualizations were code-specific while others were more high-level (Velázquez-Iturbide and

Pérez-Carrasco, 2009) (Enström, 2013) (Carlisle et al., 2005) (Zumaytis and Karnalim, 2017). However, sometimes these visualizations were more helpful for the high-level concepts but not for implementing code (Carlisle et al., 2005). Focusing on the high-level techniques rather than specific code was another common strategy (Carlisle et al., 2005) (Levitin and Papalaskari, 2002).

Finally, I explored how to teach debugging. One paper discussed how to prevent debugging and then how to trace code (Mathis, 1974). The other focused on explaining many debugging techniques and how to categorize types of bugs (Li et al., 2019).

1.4 Reflection

I found that finding sources and synthesizing their contents was slightly easier than last week but still difficult. I do not have a background in technical research so a lot of the specific data in these papers is almost meaningless to me. For example, I do not understand p values and should probably look into that more before next week so that I have a clearer understanding as I read these papers.

Thank you to one of my peer reviewers from last week who recommended seeing which papers cited the paper that I had just used, I found at least one paper with this technique. I wonder if there is a way to find the most relevant and most often cited papers, because right now I can only figure out how to sort by one or the other instead of some combination of the two.

Overall I think that this is a valuable exercise and it went smoother than last week but was still not an easy process for me.

1.5 Planning

Next week, I think I would like to explore more research on curriculum design such as how to effectively test students and how to give better feedback. Even if I develop a tool that combines many strategies to teach recursion, if the student does not get adequate feedback they may not be able to quickly understand if they are applying their knowledge correctly or not. Related to this, I would like to look into more meta research like how to effectively test different strategies and determine if they helped or not.

In addition, I would like to further look into cultivating important traits in developers such as having a methodical workflow and debugging process. I'm not

sure if there is research out there on how to cultivate a methodical workflow in students but that could be very interesting.

However, I have already looked into these topics this week and last week and still feel a little overwhelmed with the many directions to go in, so I am very open to any suggestions the reader may have for what I should explore next.

2 ACTIVITY

2.1 Introduction

Online education has the potential to allow students across geographic regions to access the same educational resources. However, a phenomenon called the "digital divide" shows that due to many factors including geographical location and socioeconomic status, many students do not have access to the resources that they need to succeed in these courses (Cullen, 2003). This is reflective of both access to equipment such as computers and high-speed internet and the resources they need to use that equipment, such as touch-typing skills. Dr. Rowena Cullen wrote in 2003 that "technology does not in itself solve social and economic discrepancies within societies, and can often exacerbate them" (Cullen, 2003). Almost twenty years later, these discrepancies are still prevalent in our society. However, online classes may even the playing field for women's access to education. Therefore, online education may make things better for some disadvantaged populations but not all.

2.2 Access to Technology

According to a study by the Pew Research Center in 2018, the percentage of U.S. households with school-age children who did not have high-speed internet at home was 15% (Anderson and Perrin, 2018). The difference between socioeconomic status was more apparent in the 2016 U.S. Census, which stated that the number of households with both a computer and a broadband Internet connection was 80% in high income homes (annual income of \$150,000 or more) but only 21% in low income homes (annual income less than \$25,000) (Ryan, 2017). Researchers at the University of Oklahoma found that there was a correlation between test scores among 5th-grade children and their access to computers and internet both in the classroom and at home (Huang and Russell, 2006). Online education may seem like a convenient option for many people, but if they do not have the resources necessary to even access their classes, they are less likely

to be successful.

2.3 Educational Access for Women

According to a study done by the University of Pennsylvania, men take more MOOCs than women (Christensen et al., 2013). However, that might not be nuanced enough to explain the benefits that women may be experiencing from online education. Researchers in Finland looked at three computer science MOOCs and compared them to in-person classes at a local university (Duran, Haaranen, and Hellas, 2020). They found that women had higher enrollment rates in the MOOCs than in the in-person classes (Duran, Haaranen, and Hellas, 2020). Therefore, while the genders may not be completely equal in online MOOCs, online education may be more approachable for women learning STEM fields.

2.4 Conclusion

In conclusion, students from certain geographical areas or from lower socioeconomic backgrounds are not given the same opportunities to succeed in online education as other students. Especially during COVID-19 when many schools are closed and students are expected to work online, it is critical that students have the hardware and access to high-speed internet that they need to log on to the classroom every day. For these students, technology is actually widening the "digital divide" and making it even more difficult for them to succeed than ever before. However, one study may show that women are more likely to take STEM classes online than in person, which eventually may make it easier for women to be in STEM fields. Therefore, online education may make education more accessible for some disadvantaged populations but certainly not all.

3 REFERENCES

- [1] Anderson, Monica and Perrin, Andrew (2018). "Nearly one-in-five teens can't always finish their homework because of the digital divide". In: *Pew Research Center* 26.
- [2] Carlisle, Martin C., Wilson, Terry A., Humphries, Jeffrey W., and Hadfield, Steven M. (2005). "RAPTOR: A Visual Programming Environment for Teaching Algorithmic Problem Solving". In: *Proceedings of the 36th SIGCSE Technical Symposium on Computer Science Education*. SIGCSE '05. St. Louis, Missouri, USA: Association for Computing Machinery, pp. 176–180. ISBN:

1581139977. DOI: [10.1145/1047344.1047411](https://doi.org/10.1145/1047344.1047411). URL: <https://doi.org/10.1145/1047344.1047411>.
- [3] Christensen, Gayle, Steinmetz, Andrew, Alcorn, Brandon, Bennett, Amy, Woods, Deirdre, and Emanuel, Ezekiel (2013). "The MOOC phenomenon: Who takes massive open online courses and why?" In: *Available at SSRN* 2350964.
 - [4] Cullen, Rowena (2003). "The digital divide: a global and national call to action". In: *The electronic library*.
 - [5] Dimitrov, Dimiter M and Rumrill Jr, Phillip D (2003). "Pretest-posttest designs and measurement of change". In: *Work* 20.2, pp. 159–165.
 - [6] Duran, Rodrigo, Haaranen, Lassi, and Hellas, Arto (2020). "Gender Differences in Introductory Programming: Comparing MOOCs and Local Courses". In: *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, pp. 692–698.
 - [7] Enström, E. (2013). "Dynamic programming - Structure, difficulties and teaching". In: *2013 IEEE Frontiers in Education Conference (FIE)*, pp. 1857–1863. DOI: [10.1109/FIE.2013.6685158](https://doi.org/10.1109/FIE.2013.6685158).
 - [8] Hattie, John and Timperley, Helen (2007). "The Power of Feedback". In: *Review of Educational Research* 77.1, pp. 81–112. DOI: [10.3102/003465430298487](https://doi.org/10.3102/003465430298487). eprint: <https://doi.org/10.3102/003465430298487>. URL: <https://doi.org/10.3102/003465430298487>.
 - [9] Huang, Jie and Russell, Susan (2006). "The digital divide and academic achievement". eng. In: *Electronic library* 24.2, pp. 160–173. ISSN: 0264-0473.
 - [10] Lau, Wilfred WF and Yuen, Allan HK (2009). "Exploring the effects of gender and learning styles on computer programming performance: implications for programming pedagogy". In: *British Journal of Educational Technology* 40.4, pp. 696–712.
 - [11] Levitin, Anany and Papalaskari, Mary-Angela (Feb. 2002). "Using Puzzles in Teaching Algorithms". In: *SIGCSE Bull.* 34.1, pp. 292–296. ISSN: 0097-8418. DOI: [10.1145/563517.563456](https://doi.org/10.1145/563517.563456). URL: <https://doi.org/10.1145/563517.563456>.
 - [12] Li, Chen, Chan, Emily, Denny, Paul, Luxton-Reilly, Andrew, and Tempero, Ewan (2019). "Towards a Framework for Teaching Debugging". In: *Proceedings of the Twenty-First Australasian Computing Education Conference. ACE '19*. Sydney, NSW, Australia: Association for Computing Machinery,

- p. 79–86. ISBN: 9781450366229. DOI:
- [10 . 1145 / 3286960 . 3286970](https://doi.org/10.1145/3286960.3286970)
- . URL:
- <https://doi.org/10.1145/3286960.3286970>
- .
- [13] Mathis, Robert F. (Jan. 1974). “Teaching Debugging”. In: *SIGCSE Bull.* 6.1, pp. 59–63. ISSN: 0097-8418. DOI: [10 . 1145 / 953057 . 810443](https://doi.org/10.1145/953057.810443). URL: <https://doi.org/10.1145/953057.810443>.
 - [14] McGill, Monica M. and Decker, Adrienne (2020). “A Gap Analysis of Statistical Data Reporting in K-12 Computing Education Research: Recommendations for Improvement”. In: *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*. SIGCSE ’20. Portland, OR, USA: Association for Computing Machinery, pp. 591–597. ISBN: 9781450367936. DOI: [10 . 1145 / 3328778 . 3366842](https://doi.org/10.1145/3328778.3366842). URL: <https://doi.org/10.1145/3328778.3366842>.
 - [15] O’Dell, Devon H. (Feb. 2017). “The Debugging Mindset: Understanding the Psychology of Learning Strategies Leads to Effective Problem-Solving Skills.” In: *Queue* 15.1, pp. 71–90. ISSN: 1542-7730. DOI: [10 . 1145 / 3055301 . 3068754](https://doi.org/10.1145/3055301.3068754). URL: <https://doi.org/10.1145/3055301.3068754>.
 - [16] Rohrer, Doug (2012). “Interleaving Helps Students Distinguish among Similar Concepts”. In: *Educational psychology review*. 24.3, pp. 355–367. ISSN: 1040-726X.
 - [17] Ryan, Camille (2017). *Computer and Internet Use in the United States: 2016*. Vol. ACS-39. Washington, DC: U.S. Census Bureau.
 - [18] Terhart, Ewald (2011). “Has John Hattie really found the holy grail of research on teaching? An extended review of Visible Learning”. In: *Journal of Curriculum Studies* 43.3, pp. 425–438. DOI: [10 . 1080 / 00220272 . 2011 . 576774](https://doi.org/10.1080/00220272.2011.576774). eprint: <https://doi.org/10.1080/00220272.2011.576774>. URL: <https://doi.org/10.1080/00220272.2011.576774>.
 - [19] Thomas, Lynda, Ratcliffe, Mark, Woodbury, John, and Jarman, Emma (Feb. 2002). “Learning Styles and Performance in the Introductory Programming Sequence”. In: *SIGCSE Bull.* 34.1, pp. 33–37. ISSN: 0097-8418. DOI: [10 . 1145 / 563517 . 563352](https://doi.org/10.1145/563517.563352). URL: <https://doi.org/10.1145/563517.563352>.
 - [20] Velázquez-Iturbide, J. Ángel and Pérez-Carrasco, Antonio (July 2009). “Active Learning of Greedy Algorithms by Means of Interactive Experimentation”. In: *SIGCSE Bull.* 41.3, pp. 119–123. ISSN: 0097-8418. DOI: [10 . 1145 / 1595496 . 1562917](https://doi.org/10.1145/1595496.1562917). URL: <https://doi.org/10.1145/1595496.1562917>.

- [21] Zumaytis, Sofriesilero and Karnalim, Oscar (2017). "Introducing an educational tool for learning branch & bound strategy". In: *Journal of Information Systems Engineering and Business Intelligence* 3.1, pp. 8–15.